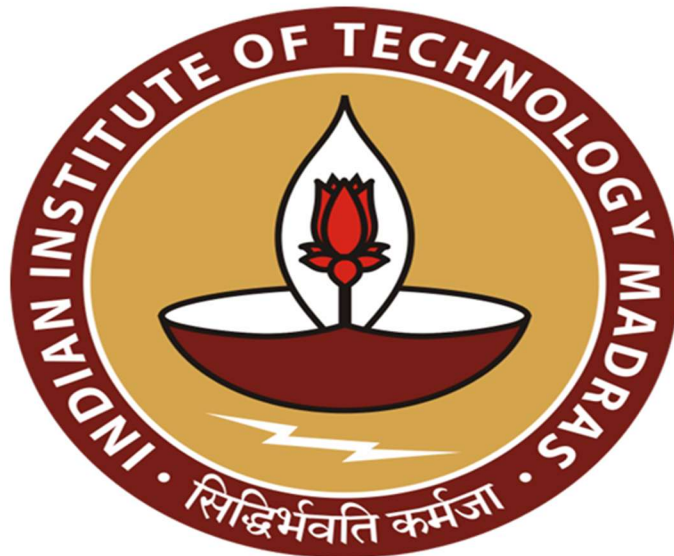


Project Report: Trekking Management Application

Student Details

- **Name:** Paras
- **Student ID / Roll No:** 23F2003850
- **Email:** 23f2003850@ds.study.iitm.ac.in



IITM Online BS Degree Program,
Indian Institute of Technology, Madras, Chennai
Tamil Nadu, India, 600036

1. Project Details & Problem Statement

Problem Statement: Adventure organizations frequently face difficulties in coordinating trekking activities using manual methods like spreadsheets and phone calls. This traditional approach results in inefficient booking management, higher risks of overbooking, and poor tracking of user trek history.

Objective: To build a comprehensive web-based Trekking Management Application that streamlines the entire process. The system must support three distinct user roles:

- **Admin:** Can create treks, assign staff, approve staff registrations, and view all system statistics.
- **Trek Staff:** Can update the status of their assigned treks (e.g., Open, Closed, Completed) and view their registered participants.
- **Users (Trekkers):** Can browse available open treks, book slots, and view their personal trekking history.

2. Approach to the Problem Statement

To build a secure, efficient, and scalable application according to the guidelines, I adopted a monolithic architecture using Flask and SQLite.

- **Unified Authentication:** Instead of creating separate database tables for Admins, Staff, and Users, I created a single User table with a role attribute. This allowed me to use Flask-Login to manage all sessions seamlessly while restricting access via role-based backend checks.
- **Data Isolation:** To keep the User table clean, I created a separate Staff Profile table linked via a one-to-one relationship. This stores staff-specific details without cluttering regular user data.
- **Programmatic Database Generation:** To adhere to project constraints, the SQLite database is created automatically when the application runs, and a default system admin is securely injected into the database using hashed passwords.
- **Dynamic UI:** I utilized a master base.html template using Jinja2 and Bootstrap 5. The navigation bar dynamically alters its links based on whether the logged-in user is an Admin, Staff member, or Trekker.

3. Frameworks and Libraries Used

- **Backend Framework:** Flask (v3.0.2)
- **Database:** SQLite
- **ORM:** Flask-SQLAlchemy (v3.1.1)
- **Authentication:** Flask-Login (v0.6.3)
- **Security:** Werkzeug (Password Hashing)
- **Frontend:** HTML5, Jinja2, CSS, Bootstrap 5

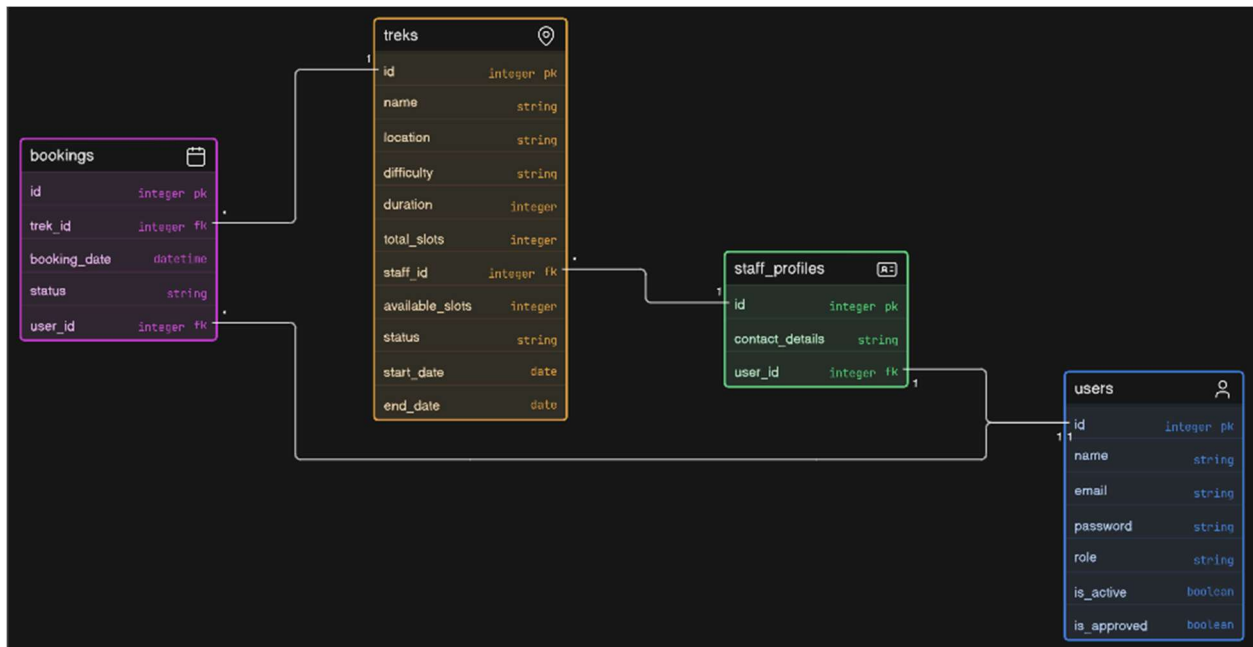
4. Entity-Relationship (ER) Diagram Schema

Tables & Attributes:

- **Users:** id (PK), name, email, password, role, is_active, is_approved
- **StaffProfile:** id (PK), contact_details, user_id (FK)
- **Treks:** id (PK), name, location, difficulty, duration, total_slots, available_slots, status, start_date, end_date, staff_id (FK)
- **Bookings:** id (PK), booking_date, status, user_id (FK), trek_id (FK)

Relationships:

1. **User to StaffProfile:** One-to-One
2. **User to Booking:** One-to-Many
3. **StaffProfile to Trek:** One-to-Many
4. **Trek to Booking:** One-to-Many



5.Application Routes (Endpoints)

While a strict REST API returning JSON was optional, the application utilizes standard HTTP endpoints to securely manage authentication, treks, bookings, and user operations.

Authentication Routes:

- GET/POST /register – Register a new Trekker or Staff account.
- GET/POST /login – Authenticate users and initiate a secure session.
- GET /logout – Terminate the current user session.

Admin Routes:

- GET /admin/dashboard – View platform statistics, including total treks, users, and staff.
- GET/POST /admin/create_trek – Create and assign a new trek to an approved staff member.
- GET/POST /admin/edit_trek/<trek_id> – Modify trek details, schedule, capacity, and assigned staff.
- GET /admin/treks – View and search all registered treks.

- POST /admin/delete_trek/<trek_id> – Delete a trek and its associated bookings.
- GET /admin/users – View and search all registered users.
- GET /admin/manage_staff – View all staff members awaiting approval.
- POST /admin/approve_staff/<user_id> – Approve a staff registration request.
- POST /admin/toggle_user/<user_id> – Activate or deactivate (blacklist) a user account.

Staff Routes:

- GET /staff/dashboard – View all treks assigned to the logged-in staff member.
- GET/POST /staff/manage_trek/<trek_id> – Manage assigned trek details, update trek status, and view participant bookings.

Trekker (User) Routes:

- GET /user/dashboard – View all available open treks.
- POST /user/book_trek/<trek_id> – Book a trek by securely reserving an available slot.
- GET /user/history – View the user's complete trek booking history.

6. AI/LLM Declaration

Declaration of AI Usage: I utilized an AI/LLM assistant (Google Gemini) to aid in the development of this project. The extent of AI/LLM usage is around 10–15%, strictly limited to code suggestions and documentation formatting. Specifically, the AI was used for:

1. **Database Models:** Assisting in writing the initial SQLAlchemy ORM model definitions and establishing relationships.
2. **Documentation:** Creating API documentation samples and structuring formatting.

3. **Code Quality:** Improving variable naming consistency and readability across the codebase.

Self-Attestation: All final implementation logic, debugging, and system integration were completed manually by me. While AI was used for scaffolding and formatting, I fully understand the architecture of the submitted code and am fully capable of explaining the backend routing, database mechanics, and frontend integration utilized within this application.

7. Video Presentation Link

Google Link:

https://drive.google.com/file/d/1niPRxpJwetnTibdL_U0VccsssWPFDBRJ/view?usp=sharing